



**TESTING
&
PENTESTING**

WAAR STAAN WE

Workshop	Thema	Deliverable
WS01	Wet- & regelgeving, NEN-7510, audit mindset	Begrippenkader
WS02	Hardening CI/CD pipeline (CodeQL, Dependabot, SBOM)	Beveiligde pipeline
WS03	Healthcare Risk Assessment (risicomatrix, bow-tie)	Risicorapport
WS04(+B)	Compliance Scanning (CVE/CVSS, Snyk, security backlog)	Scanrapport + backlog + test results
WS05	Secure Coding & Privacy by Design	Logging + tests
WS06	Formeel auditrapport	Definitief auditrapport

LEERDOELEN

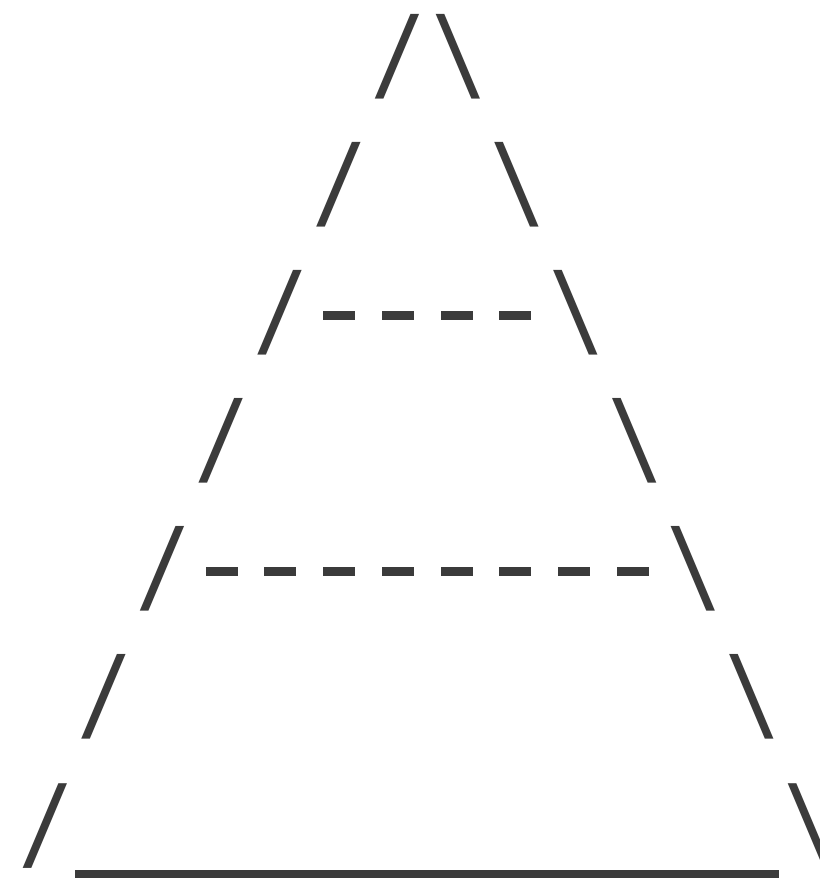
Na deze workshop kun je...

- ✓ De testpiramide uitleggen en voor elk niveau de bijdrage aan NEN-7510:2024-2 compliance benoemen.
- ✓ Security-relevante unit tests schrijven in Java voor authenticatie, toegangsbeheer en logging.
- ✓ Integratietests opzetten die toegangscontrole en API-beveiliging eind-tot-eind valideren.
- ✓ DAST uitvoeren met OWASP ZAP en de output koppelen aan een bevinding in de security backlog.
- ✓ Een pentest uitvoeren volgens de OWASP Testing Guide-methodologie en bevindingen vastleggen in een compliance-ready rapportageformat.

Waarom testen (volgens NEN-7510)

NEN-7510:2024-2 Control	Verplichting
8.25 Veilige ontwikkelcyclus	Security testing in elke SDLC-fase
8.28 Veilig coderen	Aantoonbare testaanpak als onderdeel van coding standards
8.29 Beveiligingstests	Formele security tests vóór productie-release
8.8 Kwetsbaarheidsbeheer	Actief opsporen én verifiëren van kwetsbaarheden
8.33 Testdata	Geen productiedata (patiëntgegevens) in test-/acceptatieomgeving

Test piramide



← Pentest / DAST (traag, duur, periodiek)

← Integratietests (API, access control)

← Unit tests (snel, goedkoop, continu)

Niveau	Snelheid	Kosten	Wanneer
Unit tests	Seconden	Laag	Bij elke commit (CI)
Integratietests	Minuten	Middel	Bij elke PR / nightly
DAST / pentest	Uren–dagen	Hoog	Pre-release / periodiek

Waarom testen (volgens NEN-7510)

ICT systemen in het algemeen

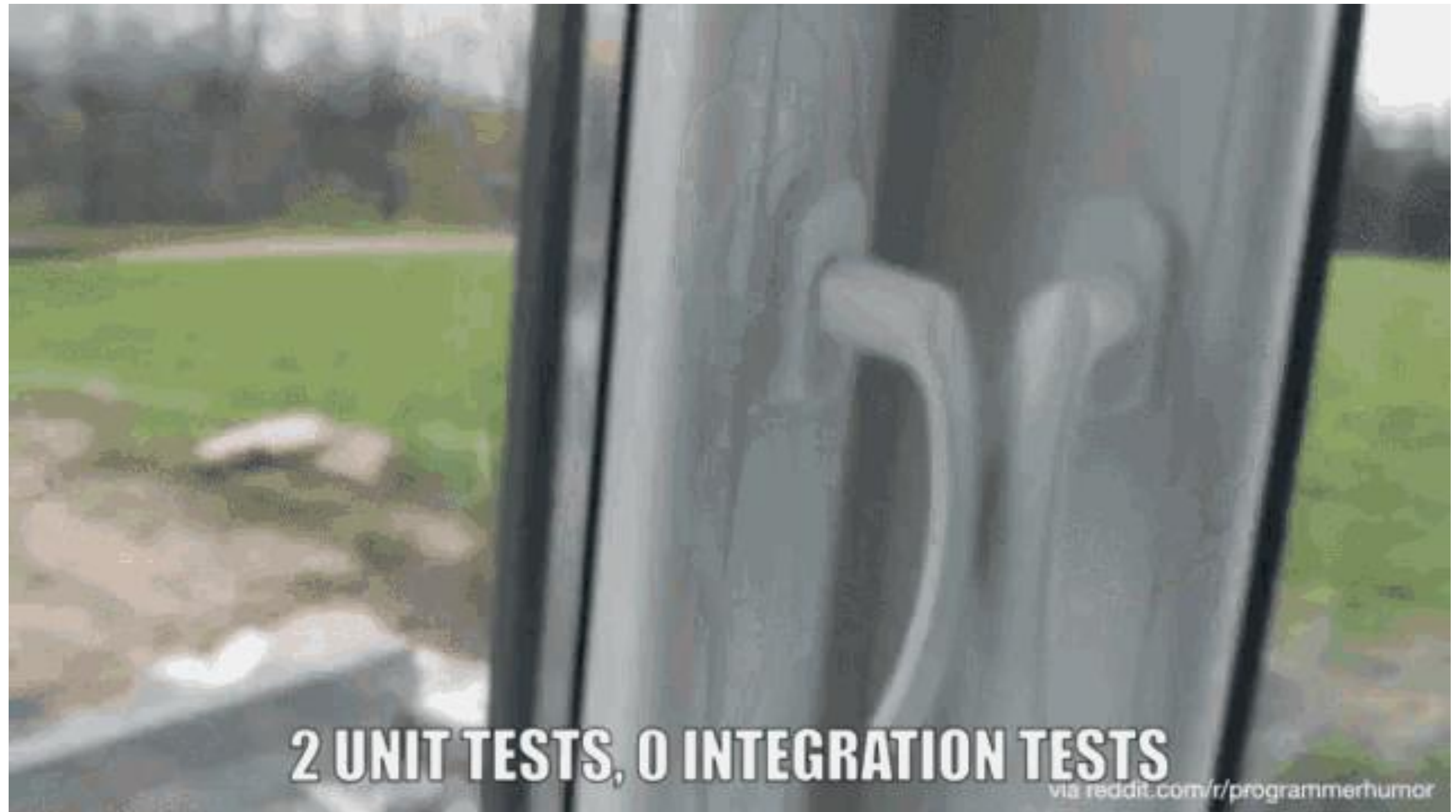
- Consequenties bij niet volgen regelgeving
 - Juridisch, reputatie, financieel (schadeclaims)

Dit project in het bijzonder

- Medische gegevens
 - Dossiers, diagnoses, behandelplannen
- Zijn bijzondere persoonsgegevens, erg gevoelig
 - Manier van verwerking niet overgelaten aan partijen

Testvormen

Techniek	Wat testen?	Toegang code	Wanneer	NEN-7510
Unit test	Één klasse/methode in isolatie	Volledig	Elke commit	8.25, 8.28, 8.29
Integratietest	Samenwerking componenten (API, DB)	Volledig	Elke PR	8.25, 8.29
SAST	Broncode statisch op kwetsbaarheidspatronen	Volledig	CI-pipeline	8.8, 8.29
SCA	Third-party dependencies op CVE's	(pom.xml)	Continu	8.8
DAST	Draaiende applicatie, extern aanvallen	Black-box	Staging/pre-release	8.29
Pentest	Volledig systeem, gecombineerde technieken	Grey-box	Periodiek / pre-release	8.8, 8.29



UNIT TESTING

Unit tests | What & Why

Een unit test valideert één geïsoleerde eenheid code (meestal methode) met een vooraf bekende invoer en verwachte uitvoer.

- In een security-context test je:
 - Authenticatie-logica (wordt een fout wachtwoord geweigerd?)
 - Toegangsbeheer (mag rol X actie Y uitvoeren?)
 - Input-validatie (wordt SQL-injectie tegengehouden?)
 - Logging (wordt een gevoelige actie gelogd met de 5 W's?)

Tooling voor OpenMRS/Java:

- JUnit 5 — testframework
- Mockito — mock dependencies (database, services) weg
- JaCoCo — meet code coverage als CI-artifact

NEN-7510 8.28: aantoonbare testaanpak is onderdeel van veilig coderen.

Unit tests | Voorbeeld toegangsbeheer

// Test: een gebruiker zonder EDIT_PATIENT-rol mag geen patiënt opslaan

@Test

```
void savePatient_withoutPermission_throwsAccessDeniedException() {
```

// Arrange

```
when(context.hasPrivilege("Edit Patient")).thenReturn(false);
```

```
Patient patient = new Patient();
```

// Act & Assert

```
assertThrows(AccessDeniedException.class,  
    () -> patientService.savePatient(patient));
```

```
}
```

// Test: een gebruiker MET de juiste rol mag wel opslaan

@Test

```
void savePatient_withPermission_succeeds() {
```

```
when(context.hasPrivilege("Edit Patient")).thenReturn(true);
```

```
patientService.savePatient(new Patient());
```

```
verify(patientDAO).savePatient(any(Patient.class));
```

```
}
```

Unit tests | Voorbeeld audit logging

```
@Test
void viewPatient_logsAccessWithFiveWs() {
    // Arrange: vang Log-berichten op
    ListAppender<ILoggingEvent> appender = new ListAppender<>();
    Logger logger = (Logger) LoggerFactory.getLogger(PatientService.class);
    logger.addAppender(appender);
    appender.start();

    // Act
    patientService.getPatient(42);

    // Assert: controleer dat het logbericht de 5 W's bevat
    assertThat(appender.list)
        .anyMatch(e -> e.getMessage().contains("PATIENT_VIEW")
            && e.getMessage().contains("patientId=42"));
}
```

Unit test vs. Integration test

INTEGRATION TESTING

Integration tests | What & Why

Een integratietest valideert de samenwerking tussen meerdere componenten: API-laag, serviceklasse én database, of externe service-integratie.

- In een security-context test je:
 - Werkt de REST API-autorisatie correct eind-tot-eind? (niet alleen in de service-klasse)
 - Geeft een unauthenticated request een HTTP 401 terug — niet 200?
 - Bevat de API-response geen velden die niet teruggegeven mogen worden (over-exposure)?

Tooling voor OpenMRS/Java:

- Spring MockMvc / RestAssured — HTTP-calls testen zonder echte server
- Testcontainers — draait een echte MySQL in Docker voor de test
- H2 in-memory database — snelle integratietests zonder Docker

NEN-7510 8.29: beveiligingstests moeten aantoonbaar zijn uitgevoerd vóór productie-release.

Integration tests | Voorbeeld API integratie *avans* hogeschool

```
@Test
void getPatient_unauthenticated_returns401() throws Exception {
    mockMvc.perform(get("/ws/rest/v1/patient/42"))
        .andExpect(status().isUnauthorized());
}

@Test
void getPatient_asNurse_returnsPatientData() throws Exception {
    mockMvc.perform(get("/ws/rest/v1/patient/42")
        .with(httpBasic("nurse_user", "password")))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.uuid").exists())
        .andExpect(jsonPath("$.auditInfo").doesNotExist()); // privacy check
}
```

Toont aan:

- Unauthenticated toegang geblokkeerd (NEN-7510 8.5)
- Gevoelige velden (auditInfo) niet in response (NEN-7510 8.3 + Privacy by Design)



DAST

Dynamic
Application
Security
Tests

DAST | What & Why

Dynamic Application Security Testing

- Aanvallen op een *draaiende* applicatie, zonder toegang tot broncode. Simuleert een externe aanvaller.

Tools: OWASP ZAP (open source)

- Spider: automatisch alle endpoints ontdekken
- Active Scan: bekende aanvalspatronen uitvoeren (XSS, SQLi, CSRF, etc.)
- Integreerbaar als GitHub Actions-stap
 - `zaproxy/action-full-scan`

Beperking: DAST vindt oppervlakkige kwetsbaarheden goed, maar mist logica-fouten (IDOR, privilege escalation)

- daarvoor is een pentest.

Wanneer inzetten: staging-omgeving, na deployment van een nieuwe versie.

NEN-7510 8.29: DAST-output als CI-artifact = aantoonbaar testbewijs.



PENTESTING

Penetration testing | What & Why

Een **geautoriseerde**, gesimuleerde aanval op een systeem.

Verschil met DAST:

- DAST = geautomatiseerd, breed, snel
- Pentest = handmatig, diepgaand, gevoel voor context (IDOR, business logic flaws, chained exploits). Vindt exploits voordat een echte crimineel dit doet.

Let op: toestemming verplicht

- Altijd schriftelijke toestemming vastleggen vóór je test op een productieomgeving. Zonder toestemming: Computervrede breuk (Sr art. 138a/b).

Techniek	Wat wordt getest	Wanneer	Toegang tot code
SAST	Broncode op kwetsbaarheden	Tijdens ontwikkeling	Ja (white-box)
DAST	Draaiende applicatie, black-box aanval	Na deployment	Nee (black-box)
SCA	Third-party dependencies op CVE's	Continu / CI	Ja (pom.xml)
Pentest	Volledig systeem, gecombineerde technieken	Periodiek / pre-release	Grey-box

PENTESTING → NEN-7510

Pentest-activiteit	NEN-7510:2024 Control	Toelichting
Pentest uitvoeren	8.8 – Beheer van technische kwetsbaarheden	Actief testen als beheermaatregel
Pentest uitvoeren	8.29 – Beveiligingstests tijdens ontwikkeling	Aantoonbaar testproces documenteren
Bevindingen documenteren	5.12 – Classificatie van informatie	Pentestrapport als vertrouwelijk document
Hertest na mitigatie	8.29 – Verificatie van beveiligingscontroles	Aantonen dat mitigatie werkt

PENTEST METHODOLOGIE (OWASP TESTING GUIDE)

Veel overeenkomst met Cyber Kill Chain

- **Fase 1: Planning & Scoping**
 - Doelen, scope, regels, toestemming, tijdlijn
- **Fase 2: Reconnaissance**
 - Technologiestack, versies, endpoints, gebruikers (OSINT)
- **Fase 3: Testing & Exploitation**
 - Kwetsbaarheden testen, PoC uitvoeren, impact bepalen. Focus op OWASP top 10.
- **Fase 4: Post-exploitation**
 - Wat is de maximale impact? Lateral movement? (maar blijf binnen scope van afspraak)
- **Fase 5: Reporting**
 - Bevindingen documenteren, CVSS scores, mitigaties adviseren

PENTEST | RAPPORTAGE BEVINDINGEN (VOORBEELD)

BEVINDING #[nummer]	
Titel:	[Korte beschrijving, bijv. "Reflected XSS in zoekparameter /openmrs/ws/rest/v1/patient"]
Ernst:	Critical / High / Medium / Low
CVSS v3.1:	[score] (AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
OWASP Cat.:	A03 – Injection / A07 – Auth failures / ...
NEN-7510:	8.29 – Beveiligingstests
BESCHRIJVING: [Wat is de kwetsbaarheid? Waarom is dit een risico?]	
REPRODUCEERSTAPPEN: 1. Log in als gebruiker met rol "Nurse" 2. Navigeer naar: GET /openmrs/ws/rest/v1/patient?q=<script> 3. Observeer dat script wordt uitgevoerd in de browser	
BEWIJS:	[Screenshot / HTTP-request-response / ZAP-alert]
IMPACT:	[Wat kan een aanvaller bereiken?] Bijv.: "Aanvaller kan sessie-cookies stelen van ingelogde verpleegkundigen → toegang tot EMR"
MITIGATIE:	[Concrete aanbeveling] Bijv.: "Pas output encoding toe via OWASP ESAPI: ESAPI.encoder().encodeForHTML(input)"
STATUS:	Open / In behandeling / Geverifieerd opgelost

TESTS & NEN-7510 LINK

Testactiviteit	Aantoonbaar bewijs	NEN-7510:2024-2 Control(s)
Unit tests (auth, logging)	Test-run in CI, JaCoCo-rapport	8.25, 8.28, 8.29
Integratietests (API, RBAC)	Test-run in CI, MockMvc-output	8.25, 8.29
SAST (CodeQL/Snyk)	CI-artifact, security tab GitHub	8.8, 8.29
SCA (Dependabot/Snyk)	Dependabot-alerts, SBOM	8.8
DAST (OWASP ZAP)	ZAP-rapport als CI-artifact	8.29
Pentest + hertest	Pentestrapport + hertestbewijs	8.8, 8.29, 5.12

NEN-7510 8.33 | Testdata & privacy

Regel: gebruik nooit echte patiëntgegevens in test- of acceptatieomgevingen.

Alternatieven

- Synthea — synthetische patiëntdatasets voor FHIR/HL7 (open source)
- Faker / Java Faker — willekeurige persoonsgegevens in unit tests
- OpenMRS demo data — de ingebouwde testset van OpenMRS zelf

Is een **compliance-verplichting**

- NEN-7510:2024-2 A.8.33: testdata moet worden geselecteerd, beschermd en beheerd
- AVG art. 5 lid 1b: persoonsgegevens mogen niet verder worden verwerkt dan waarvoor ze verzameld zijn
- Testomgeving != productie → verwerkingsdoel verschilt → geen productiedata in test

NEN-7510 8.33 | Testdata & privacy



Kern

- Testen is de schakel tussen "we denken dat het veilig is" en "we kunnen aantonen dat het veilig is."

Wat wanneer?

- Unit tests → continue validatie van security-logica (elke commit)
- Integratietests → eind-tot-eind validatie van toegangscontrole (bij een PR)
- SAST & SCA → geautomatiseerde scanning in de CI-pipeline
- DAST → geautomatiseerde aanval op staging (pre-release)
- Pentest → diepgaande handmatige validatie (periodiek + vóór audit)

Koppeling aan de eindopdracht:

- Unit- en integratietests → bewijs voor rubricelement D (PoC)
- DAST & pentest → bewijs voor rubricelement C (pentestrapportage)
- Alle testoutput bewaard in GitHub → bewijs voor rubricelement E (auditrapport)

